

北太天元

2025北太天元数模培训公开课

助力全国大学生数学建模竞赛

2025年07月20日—27日 · 线上直播



北太天元数模之星参赛经验分享

张一博

2023年北太天元数模之星



2023年全国大学生数学建模竞赛本科组全国一等奖、“北太天元数模之星”冠名奖核心成员。

擅长利用新兴计算工具进行模型的高效求解。他将带我们重温2023年B题的挑战，揭秘团队背后的建模思路、算法的选择、优化问题求解与团队协作秘籍。

➤ 算法的选择与求解(以优化问题为例)

- 1.没有万能的算法，只有最适合当前问题的算法
- 2.如何精准判断问题类型，从而选择与它最匹配的算法家族
3. 面对同一个问题，如何运用多种方法，增加结论的说服力。
- 4.对于描述动态系统的微分方程，如何选择解析解或数值解

➤ 算法的选择与求解(以优化问题为例)

优化问题的求解策略

1. 基础算法是基石：要熟练掌握遍历法、二分法、回溯法等经典方法来求解规划问题。

遍历算法是指对给定的数据结构（如数组、链表、树等）进行遍历，以便访问其中的每个元素或节点。遍历算法的目的是按照一定的规则，逐个检查或处理数据结构中的元素。

遍历法求解

遍历法核心策略：两步搜索法（由粗到精）

在竞赛中，为了兼顾计算效率和求解精度，遍历法通常采用“先粗后精”的两步策略

第一步：大范围、大步长的“粗搜索”

- 目的：定位最优解所在的大致区间。
- 做法：在参数的整个可能范围内，设置一个较大的步长进行遍历。例如，如果一个参数的取值范围是 $[1\text{mm}, 25\text{mm}]$ ，可以先以 1mm 为步长进行第一次搜索，确定大致的区间。
- 结果分析与可视化（第一张图：趋势图）：将这次搜索的结果绘制成图表。这张图的主要作用是展示目标函数随参数变化的整体趋势。

➤ 算法的选择与求解(以优化问题为例)

第二步：小范围、小步长的“精细搜索”

- 目的：在第一步找到的“潜力区间”内，进行高精度搜索，**精确地找出最优解**。
- 做法：针对上一步锁定的区间（如 $[15, 16]$ ），大幅度**缩小步长**。例如，将步长从 1mm 减小到 0.1mm 甚至更小，再次进行遍历。
- 结果分析与可视化（第二张图：放大图）：将精细搜索的结果也绘制成图。这张图是上一张图的“局部放大”，能够清晰地展示最优解的具体位置。

Step1:我们认为焦距应当介于 $[278, 282]$ 。于是，我们从 278m 到 282m 之间进行遍历。先对焦距选取步长为 0.1m，每隔一个步长选取一个焦距，利用 Matlab 对每一个焦距计算出目标函数值。我们利用 Matlab 的遍历结果如下所示：

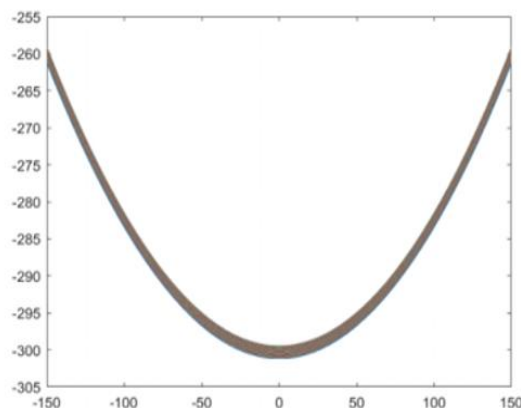


图 5.1.2.1：第一次遍历搜索结果示意图

在左图中，坐标即为我们降维后的二维坐标系，图线的含义即为：**抛物线（面）随着焦距的变化而变化的趋势。**

可以看出，随着焦距值的变化，抛物线呈下降趋势且**存在一个最接近圆的情况。**

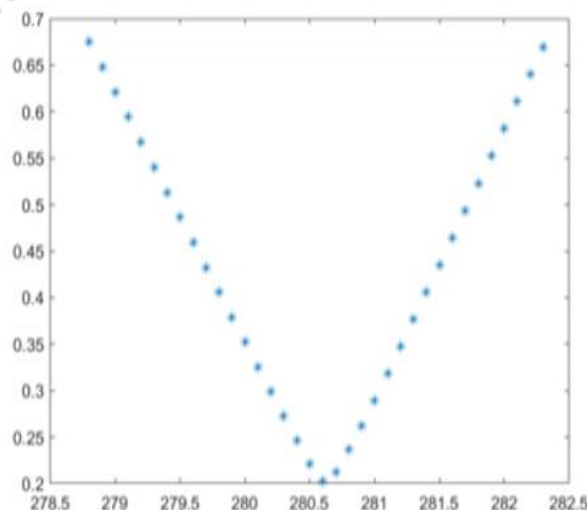


图 5.1.2.2：对最优解的第一次遍历搜索

左图为焦距在区间 $[278, 282]$ 中不断变化时，目标函数的变化趋势。

可见，在焦距为 280.6m 左右，目标函数达到最小值，最小值约为 0.2。为了确定焦距的更精确值，我们需要降低步长进行循环遍历。

➤ 算法的选择与求解(以优化问题为例)

二分法

在数学建模中，二分法是一种常用的数值方法，用于求解方程的根或函数的极值问题。其基本思想是通过不断将区间一分为二，逐步缩小搜索范围，最终找到满足精度要求的近似解。

二分法的步骤

1. 确定有根区间：首先需要确定一个包含解的区间 $[a,b]$ ，使得函数 $f(x)$ 在该区间内连续，并且 $f(a)$ 和 $f(b)$ 符号相反（即 $f(a) \cdot f(b) < 0$ ），根据介值定理，可以保证在 (a,b) 内至少存在一个实根。
2. 迭代过程：将区间 $[a,b]$ 平均分成两个子区间，取中间点 $c = \frac{a+b}{2}$ ，计算 $f(c)$ ：
 1. 如果 $f(c) = 0$ ，则找到了精确解。
 2. 否则，根据 $f(c)$ 的符号决定新的区间。如果 $f(a) \cdot f(c) < 0$ ，则新区间为 $[a,c]$ ；如果 $f(b) \cdot f(c) < 0$ ，则新区间为 $[c,b]$ 。
3. 重复步骤：对新区间重复上述步骤，每次将区间缩小一半，直到满足终止条件（如区间长度小于预设的阈值或达到预定的迭代次数）。

➤ 算法的选择与求解(以优化问题为例)

黄金法则：单调性是关键 🔑

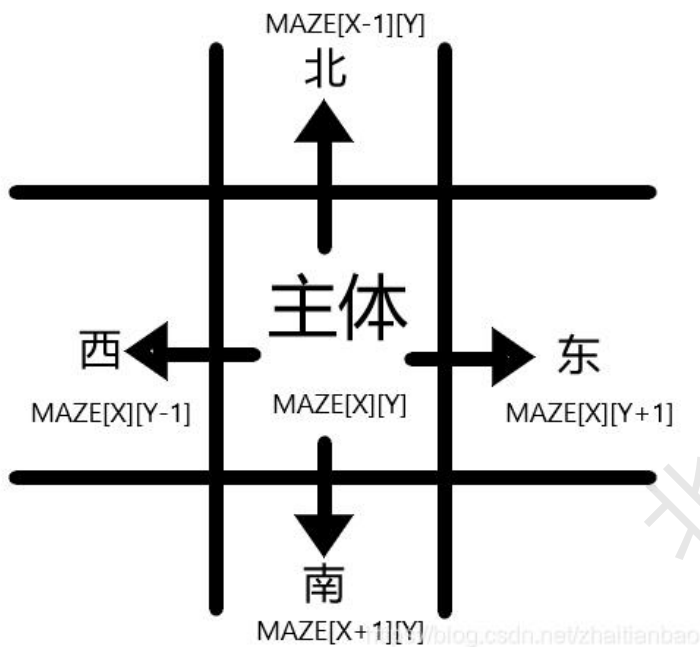
在使用二分法之前，必须满足它唯一且最重要的前提条件：**单调性**。

- **它意味着什么：** 在研究的区间内，调整的参数和你观察的结果之间，必须呈现单调递增或单调递减的关系。也就是说，当增加参数时，结果应该稳定地变大或稳定地变小，不能一会上升，一会下降。
- **更广义的理解：** 这里的“单调”不一定是指严格的数学上的单调。在建模中，它指的是内在趋势是单向的。例如，在炉温曲线和防热服这两道题目中，增加材料的厚度“总体上”会提升保温效果（最终温度降低），即便这个关系不是一条完美的直线，只要大趋势是单向的，就可以应用二分法。
- **如何在论文中说明：** 在使用该方法前，应该用一两句话解释为什么该问题满足单调性。可以通过绘制一张简单的趋势图、进行机理分析（例如，“根据热力学传导定律...”）或进行逻辑推导来证明。

➤ 算法的选择与求解(以优化问题为例)

回溯法

回溯实际上是一种试探算法，这种算法跟暴力搜索最大的不同在于，在回溯算法里，是一步一步地小心翼翼地向前试探，会对每一步探测到的情况进行评估，如果当前的情况已经无法满足要求，那么就没有必要继续进行下去，也就是说，它可以帮助我们避免走很多的弯路。



理解回溯法最直观的方式就是想象自己在走一个巨大的迷宫：

- 1.一条路走到黑 (向前试探):** 你站在一个路口，选择一条看起来可行的路，然后一直走下去。
- 2.发现是死胡同 (发现死路/约束冲突):** 你走着走着，发现前面是墙壁（此路不通），或者这条路不满足迷宫的某个要求（比如，不能经过同一个点两次）。
- 3.退回上个路口，换条路 (回溯):** 既然此路不通，你就必须**原路退回**到上一个走过的路口，然后选择一条**你之前没走过的**新路，继续试探。
- 4.重复此过程:** 不断重复这个“试探-碰壁-回溯-换路”的过程，直到你遍历了所有可能的路径，最终找到出口（或者发现根本没有出口）。这个“退回上个路口”的动作，就是**回溯**的精髓。

➤ 算法的选择与求解(以优化问题为例)

回溯法的代码框架

```
# results 用于存放所有找到的合法解
results = []

def backtrack(路径, 选择列表):
    # 1. 判断是否满足结束条件
    if 满足结束条件:
        将“路径”加入到 results
        return # 结束当前递归

    # 2. 遍历当前“选择列表”中的所有选择
    for 选择 in 选择列表:
        # 3. 剪枝操作: 判断当前选择是否合法
        if 当前“选择”不合法:
            continue # 跳过这个选择, 尝试下一个

        # 4. 做出选择
        将“选择”添加到“路径”中

        # 5. 进入下一层决策: 递归调用
        # 注意: 传入新的“选择列表”
        backtrack(路径, 新的选择列表)

        # 6. 撤销选择 (这步就是“回溯”的关键!)
        将“选择”从“路径”中移除
```

在数学建模中的应用场景

- **路径规划问题**: 例如, 在无人机 (RGV) 调度、物流配送等问题中, 寻找一条满足特定约束条件 (如时间、成本、载重限制) 的最优或可行路径。
- **组合优化问题**: 当需要从一组对象中选出一个子集, 以满足某些约束并优化某个目标时。例如, “0-1背包问题”——哪些物品应该放入背包, 才能在不超过总重量的前提下, 使总价值最大。
- **分配与调度问题**: 例如, 将一系列任务分配给不同的机器或工人, 每个任务有不同的耗时和成本, 要求找到总耗时最短或总成本最低的分配方案。

总结: 当一个问题可以被分解为一系列连续的决策步骤, 并且后续的决策依赖于前面的决策时, 回溯法就是一个非常值得考虑的强大工具。它的核心在于**“递归地向前试探”和“通过剪枝提升效率”**。

➤ 算法的选择与求解(以优化问题为例)

优化问题的求解策略

2. 智能算法作为”杀手锏”

为什么需要“智能”算法？

首先要明确，我们什么时候会放弃传统、精确的优化算法，转而使用智能算法？

当问题变得异常**复杂**时，常规方法可能会“失灵”。这里的“复杂”通常指：

- **非线性、不可导、不连续**：目标函数或约束条件充满了复杂的数学形式，传统的梯度下降等方法无法使用。
- **巨大的搜索空间**：解的可能性太多，像遍历法这样的“地毯式搜索”会直接导致计算超时。
- **多峰函数（存在大量局部最优解）**：问题像一个有很多山谷的连绵山脉，常规算法很容易陷入一个小山谷（局部最优解），而找不到整个山脉的最低点（全局最优解）。
- **“黑箱”问题**：你甚至不知道目标函数的具体数学表达式，只能通过输入一组参数，得到一个好坏的评价值。

➤ 算法的选择与求解(以优化问题为例)

赛题风格

在A、B类赛题中的路径规划、复杂调度、资源分配等问题，常常具备以上特征。此时，智能算法就成了我们的首选。它们的核心思想是：**放弃对“绝对最优”的执着，转而用一种高效的、受自然启发的策略，在有限的时间内找到一个足够好的“满意解”。**

➤ 算法的选择与求解(以优化问题为例)

1. 粒子群优化算法 (PSO): 社会学习的力量 🐦

粒子群算法 (Particle Swarm Optimization) 的灵感来源于鸟群觅食的行为。

设想这样一个场景：一群 🐦 在随机搜索 🐛。在这个区域处处分散着虫子。所有的 🐦 都不知道 🐛 最集中的地方在哪里。但它们知道自己当前距离食物有多远。那么，找到食物的最快策略是什么？

1. **参考自己**：每只鸟都会记住自己飞过的、离食物最近的那个位置（个体最优解 `pbest`）。
2. **参考群体**：所有鸟还会共享信息，知道整个鸟群中哪只鸟当前离食物最近（全局最优解 `gbest`）。

接下来，每只鸟的飞行方向和速度，就由这两个因素共同决定：**既要相信自己的历史经验，又要向当前最优秀的同伴学习。** 通过这种个体经验与社会信息的结合，整个鸟群能够高效地、协同地飞向食物源。

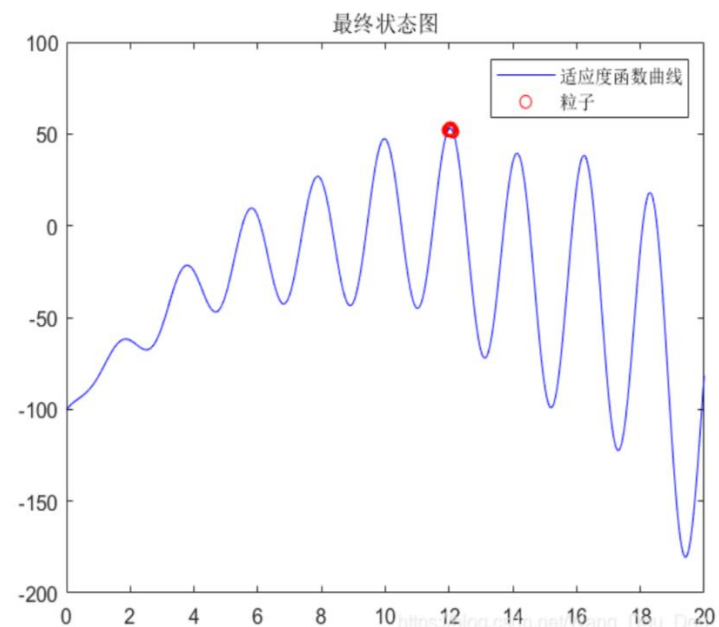
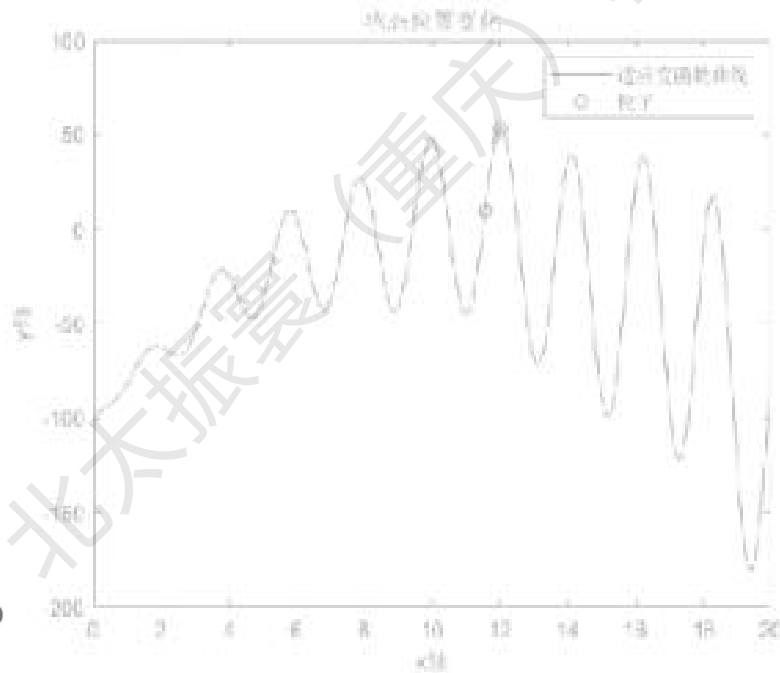
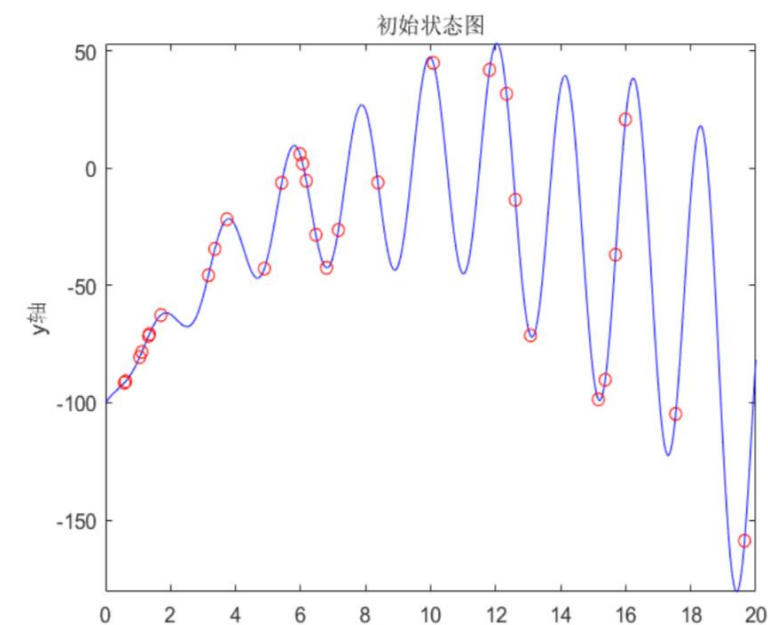
➤ 算法的选择与求解(以优化问题为例)

粒子群优化算法有什么用？

粒子群算法属于**无约束**的优化算法，是**全局搜索算法**，也是**启发式算法**。

它只能够得到全局最优的近似解，可能得不到全局最优解。

可以用在全局路径搜索、网络路由规划、寻找复杂函数的最值点等应用



➤ 算法的选择与求解(以优化问题为例)

粒子群算法模板

```
# 创建粒子群
swarm = [Particle() for _ in range(n_particles)]

# 主循环: 进行迭代
for i in range(n_iterations):
    for particle in swarm:
        # 计算当前位置的适应度
        current_fitness = objective_function(particle.position)

        # --- 更新个体最优 (pbest) ---
        if current_fitness < particle.pbest_fitness:
            particle.pbest_position = particle.position.copy()
            particle.pbest_fitness = current_fitness

        # --- 更新全局最优 (gbest) ---
        if current_fitness < gbest_fitness:
            gbest_position = particle.position.copy()
            gbest_fitness = current_fitness

    # --- 更新所有粒子的速度和位置 ---
    for particle in swarm:
        # 生成两个随机数
        r1 = np.random.rand(n_dimensions)
        r2 = np.random.rand(n_dimensions)

        # 更新速度
        cognitive_velocity = c1 * r1 * (particle.pbest_position - particle.position)
        social_velocity = c2 * r2 * (gbest_position - particle.position)
        particle.velocity = w * particle.velocity + cognitive_velocity + social_velocity

        # 更新位置
        particle.position = particle.position + particle.velocity

    # 边界处理: 确保粒子不会飞出搜索范围
    particle.position = np.clip(particle.position, x_min, x_max)
```

- 在建模中如何运作:
 - 粒子 (Particle): 每一个“粒子”都代表问题的一个潜在解。
 - 位置 (Position): 粒子的位置就是解的具体参数值。
 - 适应度 (Fitness): 通过目标函数计算出的解的优劣程度, 相当于“距离食物的远近”。
 - 迭代: 在每一次迭代中, 所有粒子根据 `pbest` 和 `gbest` 更新自己的速度和位置, 在解空间中“飞行”, 最终整个“粒子群”会聚集在全局最优解的周围。
- 优点: 算法简单, 参数较少, 收敛速度通常很快。
- 缺点: 容易过早地陷入局部最优解 (即“早熟”), 如果整个群体太快地向一个“假”的最优点聚集, 就可能找不到真正的全局最优。

➤ 算法的选择与求解(以优化问题为例)

2. 模拟退火算法 (SA): 策略性“犯错”的艺术 🔥

模拟退火算法 (Simulated Annealing) 的思想来源于固体退火过程。

将一块金属加热到极高温，再让它缓慢冷却，最终金属的内能会达到最低，结构最稳定。

- **高温状态**: 分子能量高，可以自由、剧烈地运动。
- **缓慢降温**: 分子运动逐渐减弱，但仍有一定能力跳出“不够好”的局部位置，寻找“更好”的全局最优位置。
- **低温状态**: 分子基本固定下来，整个系统达到能量最低的稳定态。

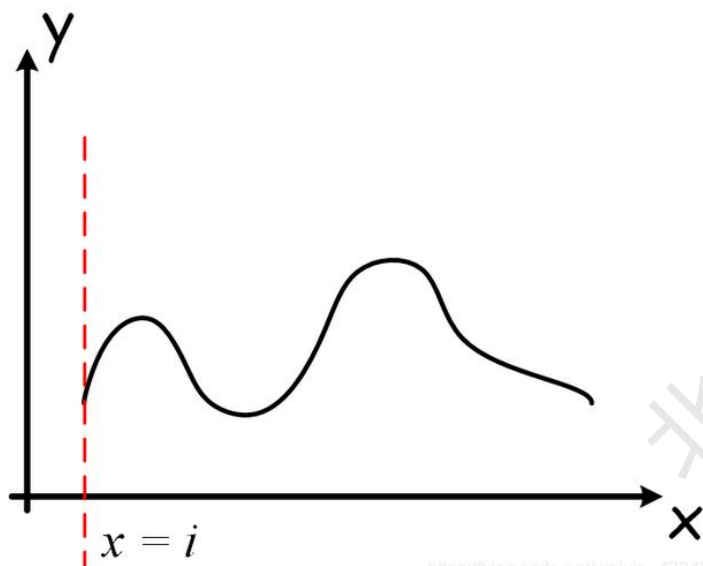
算法特点:

- 与遗传算法、粒子群优化算法等不同，模拟退火算法不属于群优化算法，**不需要初始化种群操作**。
- **收敛速度较慢**。1) 它初始温度一般设定得很高，而终止温度设定得低，这样才符合物体规律，认为物质处于最低能量平衡点；2) 它**接受更差的解**，并不是全程都在收敛的过程中。
- 温度管理（起始、终止温度）、退火速度（衰减函数）等对寻优结果均有影响。比如T的衰减速度如果太快，就会导致可能寻找不到全局最优解

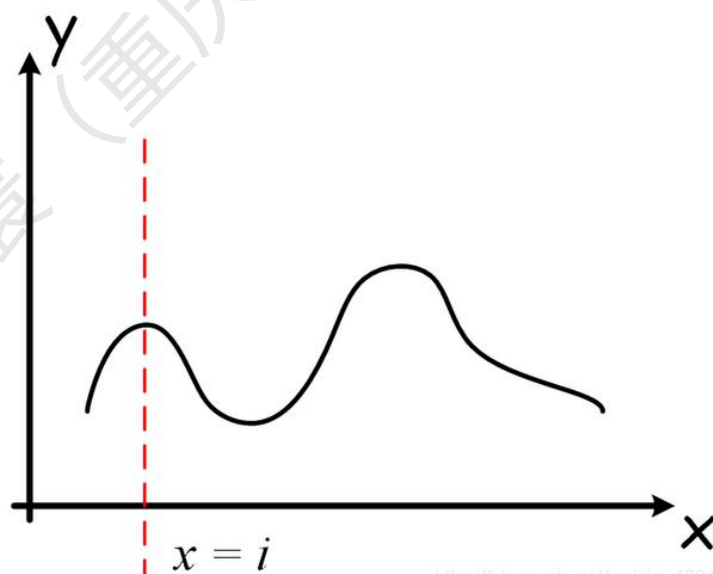
➤ 算法的选择与求解(以优化问题为例)

为什么要接受“更差的解”？

这就是模拟退火算法的精髓所在。这给了它“跳出局部最优陷阱”的能力。想象一个登山者要寻找山脉的最高点，如果他只遵循“永远向上走”的策略，他会很快卡在某个小山顶。而模拟退火就像一个允许登山者偶尔可以向上走一小步（接受一个更差的解）的策略，这使得他有机会翻过一个小山头，去寻找一个更高的山顶（全局最优解）。



https://blog.csdn.net/weixin_43848614



https://blog.csdn.net/weixin_43848614

➤ 算法的选择与求解(以优化问题为例)

模拟退火算法模板

```
# 主循环：当温度高于终止温度时，持续降温
while T > T_final:
    # 在当前温度下进行多次迭代
    for i in range(n_iterations):
        # 产生一个邻居解
        neighbor_position = get_neighbor(current_position)
        neighbor_energy = objective_function(neighbor_position)

        # 计算能量差
        delta_energy = neighbor_energy - current_energy

        # --- Metropolis 准则：决定是否接受新解 ---
        # 如果新解更好（能量更低），则总是接受
        if delta_energy < 0:
            current_position = neighbor_position
            current_energy = neighbor_energy
            # 如果这个新解比历史最优解还好，则更新历史最优
            if current_energy < best_energy:
                best_position = current_position.copy()
                best_energy = current_energy
        # 如果新解更差，则以一定概率接受（这是算法的精髓）
        else:
            acceptance_prob = math.exp(-delta_energy / T)
            if np.random.rand() < acceptance_prob:
                current_position = neighbor_position
                current_energy = neighbor_energy

    # 缓慢降温
    T *= alpha
```

• 在建模中如何运作：

该算法最核心的特点是：它有一定概率接受一个比当前解更差的解。

1. 初始化：设置一个很高的“温度” T ，并随机产生一个初始解 S 。
2. 扰动产生新解：对当前解 S 进行一个微小的扰动，产生一个邻近的新解 S' 。
3. 判断与接受：
 - 如果新解 S' 更好，则总是接受它。
 - 如果新解 S' 更差，则以一个概率 P 接受它。这个概率 P 与“温度” T 正相关——温度越高，接受坏解的概率越大；温度越低，概率越小。
4. 降温：缓慢地降低温度 T ，重复步骤2和3。

➤ 算法的选择与求解(以优化问题为例)

竞赛实战建议

• 如何选择？

- 对于**连续型优化问题**且对求解速度有一定要求时，可以优先考虑**粒子群算法**。
- 对于**组合优化问题**（如TSP、调度问题）或者怀疑存在大量局部最优解时，**模拟退火算法**跳出局部最优的能力会更有优势。

如何在论文中呈现？

- 1. 阐明选择理由：**不要只说“本文采用XX算法求解”，而要说明为什么。例如：“由于本问题的目标函数是复杂的非线性函数，且存在多个局部极值点，为避免陷入局部最优，本文选择采用全局搜索能力更强的模拟退火/粒子群算法进行求解。”
- 2. 给出关键参数：**清晰地列出使用的算法参数，如粒子群的**种群规模、惯性权重、学习因子**；模拟退火的**初始温度、降温系数、马尔可夫链长度**等。这体现了你的专业性。
- 3. 结果展示：**用**迭代收敛图**来展示算法的求解过程，即目标函数值随着迭代次数增加而逐渐优化并趋于稳定的曲线图。这比只给一个最终结果要更有说服力。

➤ 算法的选择与求解(以优化问题为例)

优化问题的求解策略

3. 微分方程的求解

微分方程——描述动态世界的语言

在数学建模中，只要一个系统的状态会随着时间或空间发生变化，我们就可以考虑使用**微分方程**来描述它。无论是物理学的热传导、力学运动，还是化学中的反应速率，甚至是经济学中的动态变化，微分方程都是我们进行机理分析和定量预测的强大武器。在一些看似与物理无关的题目（如化学问题）中，建立并求解微分方程，绝对是论文的一大亮点。

求解微分方程，首先面临一个根本性的选择：是寻求**解析解**，还是计算**数值解**？这个选择完全取决于我们建立的微分方程的形式和复杂程度。

➤ 算法的选择与求解(以优化问题为例)

1. 解析解是一个精确的、用封闭数学表达式表示的通解公式。它能完美地描述系统在任何条件下的状态。

- **何时使用：**当微分方程形式相对简单时，例如只包含一个变量的导数（如牛顿第二定律的一些情况），我们可以尝试求解解析解。

- **常用方法：**

- **拉普拉斯变换 (Laplace Transform)：**一种强大的数学工具，尤其擅长求解线性常微分方程，可以直接将微分方程转化为代数方程来求解。

2. 当方程变得复杂，无法求出精确的表达式时，我们就需要借助计算机，用一系列离散的数值点来近似描述真实的解。这是竞赛中的主流方法。

➤ 算法的选择与求解(以优化问题为例)

针对数值法

有限差分法 (Finite Difference Method)

将求解区域进行网格化离散处理，用离散点上的差商来近似代替导数，从而将偏微分方程转化为代数方程组进行求解。

•**应用场景**：这是求解**偏微分方程**的主流方法。当你的模型是抛物线型、双曲线型或椭球型微分方程时，比如经典的热传导方程，就应该首先考虑使用有限差分法。

龙格-库塔法 (Runge-Kutta Method)

一种高精度的、单步递推的算法，通过在当前点附近取多个“探测点”并对其斜率进行加权平均，来估算下一步的值。

•**应用场景**：这是求解**常微分方程**的首选方法之一，特别是当解析解难以获得时。例如，在处理类似牛顿第二定律这类问题时，龙格-库塔法是非常可靠和高效的选择。

➤ 论文写作技巧：图文并茂，化繁为简

原则：“能用图说清楚的，就别用表；能用表说清楚的，就别用大段文字。”

流程图 (Flowchart): 在模型建立部分，用一个清晰的流程图来展示你们的建模思路，让评委一目了然。

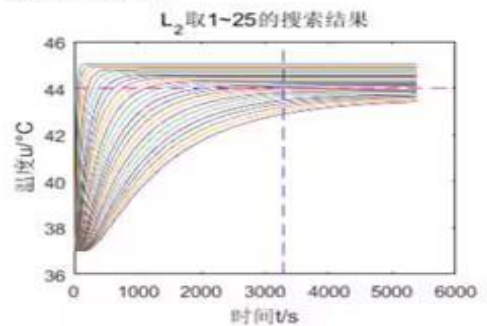
结果图 (Result Plots): 求解结果不要只给一个冷冰冰的数字。

美观性： 图表要清晰、美观，坐标轴、图例、标题一定要完整。推荐Python (Matplotlib, Seaborn), Origin 等专业软件绘图，避免直接使用 Excel 的默认样式。

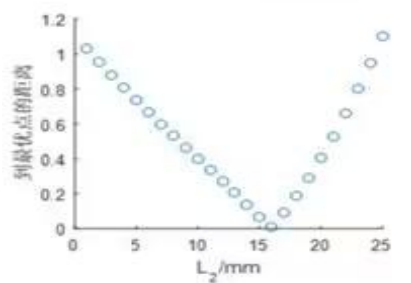
论文写作技巧：图文并茂，化繁为简

遍历搜索求解

Step1. 在区间[1mm,25mm]内以 1mm 为步长进行第一次搜索，在 Matlab 软件中得到搜索结果如下：



如左图所示，横竖虚线的交界点（工作时间 55 分钟，温度为 44 °C）即为上面所说的“临界点”。从图中可以看出，随着厚度的增加，皮肤表面到达稳态温度的速度越慢，即材料的隔热效果越好。



左图是工作时间为 55 分钟的情况下，取不同厚度时皮肤表面的温度到“临界点”温度的距离。可以看出，当厚度在 [15,16] 区间内取得距离的最小值，即最贴近“临界点”。为了确定最优厚度的精确值，我们要缩小搜索范围和搜索步长，进行二次搜索。

误差分析

为了检验模型的准确性，我们将各催化剂组合实验组的数据代入到拟定的回归方程中，得到对应的乙醇转化率和 C4 烯烃选择性的数值，并导出附件一中的数据，将拟合值和实际值进行比较，并作出残差图如下所示：

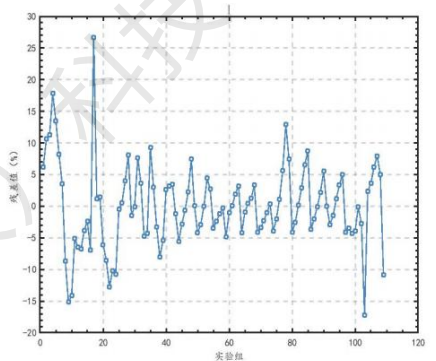


图 13: 乙醇转化率回归方程残差图

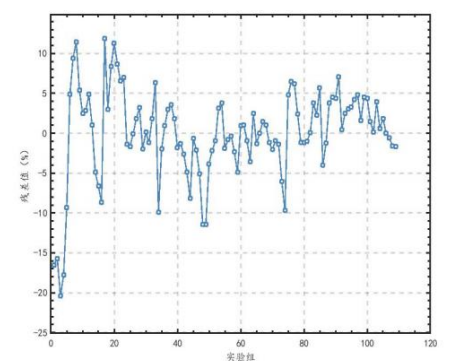


图 14: C4 烯烃选择性回归方程残差图

分析上图发现，两个回归方程的残差值均在前几组实验组中出现极大值，其余实验组中，残差值大致在±5%附近波动，稳定性较好。由于上述模型中所拟定的回归方程自变量和未知参数较多，存在一定的误差是合理的，上述残差图也有力的证明了参数值极有可能是正确的，同时说明了模型的正确性和准确性。



04 在线答疑



在线答疑